An Internship Report on

Image Captioning using a Hybrid CNN-LSTM Architecture

Submitted by

Sk. Nazirul Islam (24101106060)

Aditya Kumar Shaw (24101106065)

Sk. Abu Tahir (24101106058)

Nilanjan Chandra (24101106070)

Subhankar Barman (24101106040)

Harsh Singh (24101106019)

Arnab Bera (24101106047)

Richa Singh (24101106008)

3rd Year, B.Tech Information Technology

Under The Guidance of

Dr. Mayuk Sarkar

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

National Institute of Technology Jamshedpur

Jamshedpur – 831014

July, 2026

TABLE OF CONTENTS

1. Introduction

1.1 Project Overview

Automated image captioning sits at the intersection of Computer Vision (CV) and Natural Language Processing (NLP), and it is a genuinely difficult problem: a machine has to both understand what is happening in an image and put that understanding into a grammatically correct sentence. The goal of this internship project was to design, build, and test a deep learning model that can take an image as input and generate a descriptive caption for it. More formally, given an input image I, the model has to generate a sequence of words $Y = \{y_1, y_2, ..., y^T\}$ that best matches what a human would write for that image, i.e. it maximizes the conditional probability $P(Y|I)$.

Beyond being an interesting research problem, image captioning also has some clear practical uses. It can help assistive tools describe a scene out loud for visually impaired users, it can automatically generate tags/descriptions for large photo libraries, and it can help search engines match images against text queries.

1.2 Dataset Characteristics

To train and test the model, this project uses the Flickr8k dataset, a widely-used benchmark for image captioning. Its main characteristics are:

- Image Quantities: Exactly 8,000 high-resolution images capturing diverse environments, everyday activities, human actions, and distinct object configurations.
- Textual Descriptions: Each image is accompanied by 5 independent, human-written captions. This multi-caption strategy yields a total of 40,000 descriptions, providing the model with a rich variety of syntactic structures and lexical variations for identical visual concepts.
- Evaluation Utility: The presence of multiple reference captions mitigates human subjective bias and provides a reliable basis for statistical language evaluation metrics during the testing phase.

1.3 Proposed Architectural Approach

Instead of the older, template-based approaches to caption generation, this project uses an end-to-end Hybrid CNN-LSTM architecture built around the standard encoder-decoder setup. The visual and sequential (text) parts of the model are kept separate and are combined only near the end, as shown below:

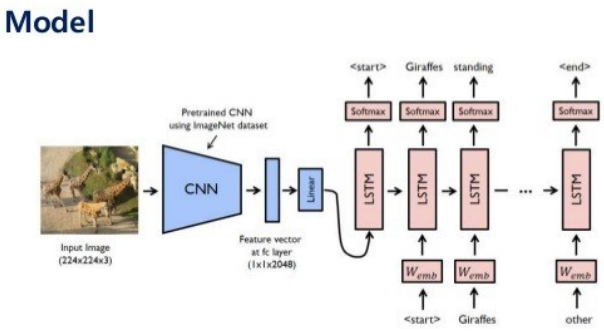

Figure: High-level view of the CNN-LSTM captioning pipeline (a pretrained CNN extracts a feature vector, which is fed to an LSTM decoder that generates the caption word by word).

The Encoder (CNN): A deep Residual Network (ResNet50), pre-trained on the ImageNet classification task, is used to "see" and interpret the image. The final classification layer is removed and the network weights are frozen, so it just acts as a fixed feature extractor: it turns the raw pixel input into a 2048-dimensional feature vector (R^{2048}).

The Decoder (LSTM): A custom Recurrent Neural Network (RNN) variant utilizing Long Short-Term Memory units is implemented to "write" the descriptive sentence. The LSTM processes the projected visual features alongside preceding text token sequence embeddings. By maintaining an internal hidden state, the decoder tracks temporal dependencies and sequence context to predict the subsequent token in the vocabulary iteratively.

1.4 Report Organization

The rest of this report is organized as follows. Section 2 covers the data preprocessing pipeline and the text/feature extraction steps used in this project. Section 3 describes the neural network architecture itself — the layers used, how they are configured, and the training setup. Section 4 presents the results: BLEU scores, training behavior, and a look at some sample outputs. Section 5 wraps up with a summary and possible directions for future work.

2. Background Study

This section covers the background concepts behind the proposed model. By reusing existing, well-established deep learning components rather than building everything from scratch, the architecture avoids the need to learn complex visual and linguistic representations from zero.

2.1 Convolutional Neural Networks (CNNs) and ResNet50

Traditional Artificial Neural Networks (ANNs) process inputs independently and struggle with the high dimensionality of raw image data. Convolutional Neural Networks (CNNs) solve this by mimicking the human visual cortex, using moving filters (kernels) to systematically scan an image and extract hierarchical features — ranging from simple edges to complex object shapes.

Training a deep CNN from scratch requires massive computational power and millions of labeled images. To bypass this computational bottleneck, this project employs Transfer Learning. Transfer learning involves taking a model that has already been trained on a massive dataset and repurposing its learned feature-extraction capabilities for a new, related task.

Specifically, this project uses ResNet50, a deep CNN pre-trained on the comprehensive ImageNet database. ResNet (Residual Network) is known for its "skip connections," which let information bypass certain layers — this is what solves the degradation problem where very deep networks otherwise struggle to learn properly.

In this project, ResNet50 acts strictly as a visual feature extractor. The model's standard classification head is removed (include_top=False), and its internal weights are frozen (trainable=False) to prevent them from updating during the training process. The image is fed into the network as a 224×224×3 pixel tensor. After passing through the convolutional layers, a global average pooling layer compresses the spatial data into a rich, dense 2048-dimensional feature vector. This vector is basically a numerical "summary" of what is in the image.

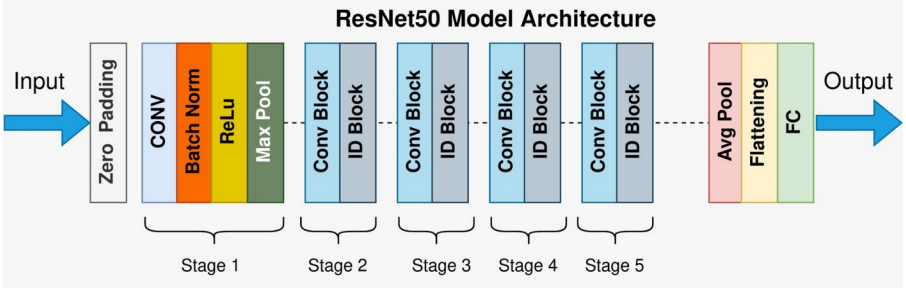

Figure 1: ResNet50 model architecture, showing the initial convolution/pooling stage followed by four stages of Conv/ID blocks and the final pooling and fully-connected layers.

2.2 Recurrent Neural Networks (RNNs) and LSTMs

While CNNs excel at processing static spatial data like images, they lack the ability to process sequential data, such as a sentence where the order of words matters. Recurrent Neural Networks (RNNs) address this by maintaining a loop within their architecture, allowing information to persist. Conceptually, this is similar to reading a book: your understanding of the current word relies heavily on the context provided by the preceding words.

However, standard RNNs suffer from "short-term memory" (the vanishing gradient problem); as sequences grow longer, they forget information from earlier in the sequence. To generate coherent, grammatically correct captions, the model requires a mechanism that can hold on to information for much longer.

This is achieved using a Long Short-Term Memory (LSTM) network, which is used in the decoder section of this project. LSTMs operate using a specialized internal memory cell regulated by three distinct "gates":

- Forget Gate: Decides what irrelevant past information should be discarded from the cell state.
- Input Gate: Determines what new, relevant information should be added to the memory.
- Output Gate: Calculates the next hidden state based on the updated memory.

By carefully managing this internal memory flow, the LSTM layer can examine the static 2048-dimensional image vector alongside the previously generated words (encoded via an Embedding layer) to accurately predict the next logical word in the caption sequence.

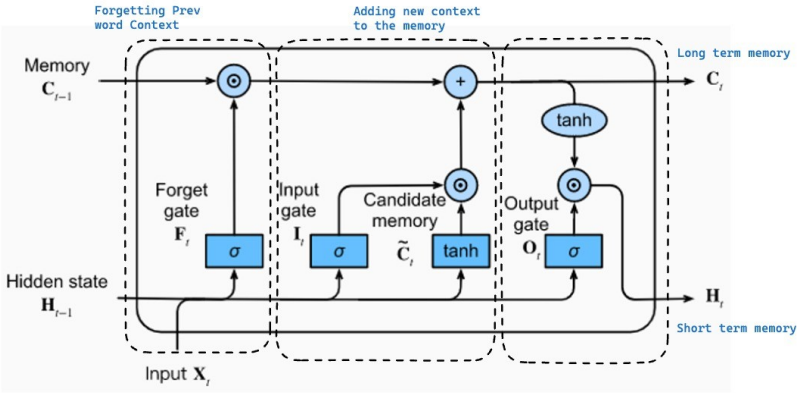

Figure 2: Internal structure of an LSTM cell, showing the forget, input, and output gates that regulate what information is kept in or discarded from the memory state.

2.3 Evaluation Metric: BLEU Score

Evaluating the quality of machine-generated text is inherently subjective. To provide a standardized, objective measure of the model's linguistic accuracy, this project utilizes the Bilingual Evaluation Understudy (BLEU) score.

In plain terms, the BLEU metric evaluates the model by mathematically comparing the machine-generated caption against the five human-written reference captions provided in the Flickr8k dataset. It calculates a score between 0.0 and 1.0 (often represented as a percentage) based on the number of overlapping words.

- BLEU-1: Measures the overlap of individual words (unigrams) between the generated text and the reference captions. This primarily evaluates whether the model successfully identified the correct objects and actions (adequacy).
- BLEU-2: Measures the overlap of two-word consecutive phrases (bigrams). This evaluates whether the model understands basic word order and local syntactic structure (fluency).

By penalizing the model for guessing words that do not appear in the human references, the BLEU score ensures that the reported accuracy reflects both the relevance and readability of the generated image descriptions.

3. Proposed Design

This section covers how the hybrid architecture was actually implemented. It starts with the data pipeline that converts raw images and text into a format the network can use, and then describes how the encoder-decoder model itself was defined using the TensorFlow/Keras framework.

3.1 System Architecture Pipeline

The proposed image captioning system operates on a dual-input structure. The visual pathway processes the image to extract distinct spatial features, while the textual pathway processes the preceding sequence of words to extract semantic context. These two independent feature vectors are subsequently fused to predict the probability distribution of the next word.

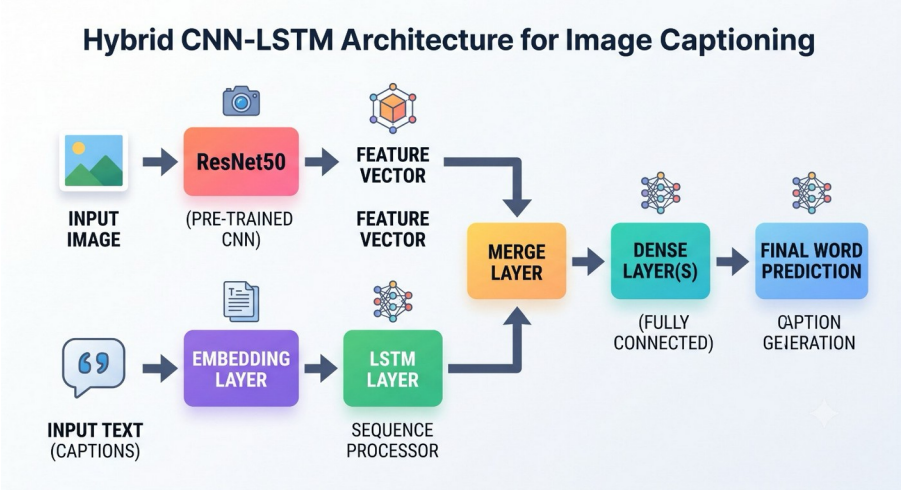

Figure 3: End-to-end architecture used in this project. The image branch (ResNet50 → feature vector) and the text branch (Embedding → LSTM) are combined in a merge layer, followed by dense layers that predict the next word.

3.2 Data Preprocessing and Tokenization

Deep learning models cannot process raw strings or raw image files directly; they require normalized, uniform numerical tensors. The preprocessing pipeline used in this project standardizes both the visual and textual data.

Visual Preprocessing: Images are loaded and strictly resized to dimensions of 224×224 pixels to match the expected input shape of the ResNet50 architecture. The image arrays are then expanded to include a batch dimension and normalized using the built-in preprocess_input function.

Textual Preprocessing: To reduce the complexity of the vocabulary, the raw human captions undergo thorough text cleaning. All text is converted to lowercase, and regular expressions are utilized to strip away punctuation and special characters (keeping only standard alphabet characters and spaces). Extraneous whitespace is also removed.

The sequence generation model also needs an explicit signal for when to start and stop generating a sentence. For this, special tags — startseq and endseq — are added to the beginning and end of every cleaned caption.

Tokenization and Padding: Following the cleaning phase, the TensorFlow/Keras Tokenizer is employed to map each unique word in the dataset's vocabulary to a specific integer index. Because neural networks require fixed-size input arrays, sentences of varying lengths cannot be fed into the model directly. To resolve this, the pad_sequences utility is applied, padding shorter sentences with zeroes so that every sequence identically matches the length of the longest caption in the dataset (max_length).

3.3 The Encoder-Decoder Model Implementation

The core model is constructed using the Keras Functional API, allowing for the creation of multi-input, non-linear neural network topologies.

3.3.1 The Visual Encoder (CNN)

The feature extraction branch relies on the pre-trained ResNet50 model. By setting include_top=False, the final classification layers responsible for labeling ImageNet classes are discarded, leaving only the convolutional base that outputs the raw mathematical representations of the image features. A global average pooling layer is applied to compress these feature maps, and the weights are frozen (trainable=False) to preserve the pre-trained knowledge.

```
# Initializing the ResNet50 Encoder
from tensorflow.keras.applications.resnet50 import ResNet50

base_model = ResNet50(weights='imagenet', input_shape=(224, 224, 3),
                      include_top=False, pooling='avg')
base_model.trainable = False
```

These features are then passed through an initial Dense and Dropout layer (to prevent overfitting) to shape the visual feature vector for the final merge.

3.3.2 The Sequence Decoder (LSTM)

Simultaneously, the sequential textual inputs are processed by the sequence decoder. The numerical word tokens are first fed into an Embedding layer, which transforms the sparse integer representations into dense semantic vectors. These embeddings are then processed by a Dropout layer and subsequently passed into the LSTM layer, which maintains the temporal memory of the sentence generated thus far.

```
# Initializing the Sequence Decoder
from tensorflow.keras.layers import Input, Embedding, Dropout, LSTM

# Text sequence feature extractor
inputs2 = Input(shape=(max_length,))
se1 = Embedding(vocab_size, 256, mask_zero=True)(inputs2)
se2 = Dropout(0.5)(se1)
se3 = LSTM(256)(se2)
```

3.3.3 The Merge Layer and Output

Once the image features and text sequence features are projected into matching dimensions, they are combined using an add layer. This fused representation contains both the visual context of the scene and the grammatical context of the current sentence. It is fed through a final fully connected Dense layer utilizing a softmax activation function to predict the probability of the next word across the entire vocabulary space.

```
# Merging Branches and Final Output
from tensorflow.keras.layers import add, Dense

# Merging both networks
decoder1 = add([fe2, se3])
decoder2 = Dense(256, activation='relu')(decoder1)
outputs = Dense(vocab_size, activation='softmax')(decoder2)
```

4. Results and Evaluation

This section presents the quantitative metrics and qualitative outputs of the trained hybrid CNN-LSTM model. The evaluation assesses both the convergence behavior during the training phase and the linguistic accuracy of the generated captions during the testing phase.

4.1 Training Performance

The model's learning phase was monitored by tracking the categorical cross-entropy loss over the training epochs. The loss function measures the discrepancy between the predicted probability distribution of the vocabulary tokens and the actual target tokens.

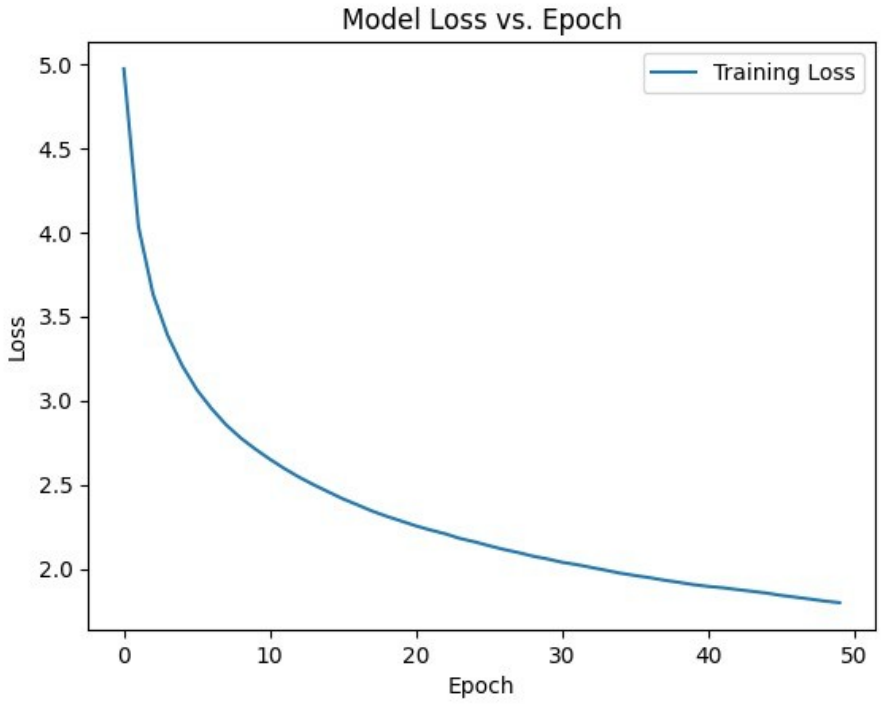

Figure 4: Training loss vs. epoch, showing the categorical cross-entropy loss decreasing steadily over 50 epochs.

As illustrated in the placeholder graph above, the training loss exhibits a steep initial decline during the first few epochs before gradually stabilizing into a steady convergence. This downward trend confirms that the network was successfully minimizing the prediction error over time, i.e. it was learning to map the R^{2048} image features to the corresponding words.

4.2 Quantitative Evaluation: BLEU Scores

To objectively measure the performance of the generated text against the Flickr8k ground-truth descriptions, the model was evaluated using the Bilingual Evaluation Understudy (BLEU) metric. The evaluation was conducted on an unseen test split to ensure the model's generalization capabilities.

Table 1: Model Evaluation Metrics on Flickr8k Test Set

Evaluation Metric	Score	Definition
BLEU-1 Score	0.5544	Measures unigram (single word) overlap, indicating accurate object and action recognition.
BLEU-2 Score	0.3321	Measures bigram (two-word phrase) overlap, indicating basic grammatical fluency and word order.

The recorded BLEU-1 score of 0.5544 demonstrates the model's strong adequacy in identifying primary subjects (e.g., "dog," "man," "running"), with more than half of the generated unigrams matching the human reference captions. The corresponding BLEU-2 score of 0.3321 indicates a reasonable grasp of local syntax, successfully pairing adjectives with nouns or verbs with subjects (e.g., "black dog," "man running"), though the expected drop from BLEU-1 to BLEU-2 reflects the added difficulty of matching consecutive word pairs rather than isolated words.

4.3 Qualitative Analysis and Practical Limitations

While statistical metrics provide a macro-level overview, qualitative visual inspection is required to understand the model's practical behavior and inherent limitations. Below are sample predictions generated by the model compared against their reference ground-truth captions.

Example 1: Correct Recognition with Simplified Description

Actual Caption: "two different breeds of brown and white dogs play on the beach"

Predicted Caption: "two dogs play on the beach"

Analysis: The model correctly identifies the core subjects ("dogs"), the setting ("beach"), and the action ("play"), producing a caption that is semantically accurate and grammatically clean. However, the

prediction is noticeably more generic than the reference: it omits the finer-grained attribute that the two animals belong to "different breeds" and does not mention their coloring ("brown and white"). This behavior is typical of the LSTM decoder, which tends to default to short, high-frequency phrasings learned from the most common sentence patterns in the training corpus rather than reproducing the more descriptive, attribute-rich phrasing a human annotator might use. The result illustrates a common precision-versus-detail trade-off: the caption is correct but conservative, capturing the gist of the scene while sacrificing some descriptive richness.

Example 2: Misclassification of Subjects and Scene Context

Actual Caption: "man and woman pose for the camera while another man looks on"

Predicted Caption: "two women are sitting on the sidewalk together and smile"

Analysis: This example exposes a clear limitation of the model. The prediction misidentifies the gender of one of the two central subjects, describing both as "women" when the reference caption specifies a "man and woman." The model also fails to register the background subject entirely ("another man looks on"), and misreads the action, describing the pair as "sitting on the sidewalk" rather than "posing for the camera." These errors are consistent with the known weaknesses of a global-average-pooled CNN encoder: compressing the entire scene into a single 2048-dimensional vector discards the spatial and fine-grained visual cues needed to reliably distinguish gender, count distinct people in the background, and disambiguate similar-looking human poses. It also reflects a dataset bias, where the decoder appears to default to the statistically common phrase "two women … smile" seen frequently during training, rather than correctly grounding its output in the specific visual evidence of this image.

5. Conclusion

5.1 Project Summary

This internship project involved designing, building, and evaluating an automated image captioning system. Using a hybrid deep learning architecture, the project brought together two normally separate areas of AI: Computer Vision and Natural Language Processing. The results show that combining a pre-trained CNN (ResNet50) for extracting visual features with an LSTM for generating text works well in practice — the system was able to learn a mapping from raw image data to coherent, human-readable

captions, which confirms that the encoder-decoder approach is a reasonable choice for this kind of multimodal task.

5.2 Internship Practical Takeaways

From a practical engineering perspective, this internship gave hands-on exposure to the full deep learning development lifecycle. Key technical competencies acquired during this project include:

- Multimodal Data Engineering: Developing an integrated preprocessing pipeline capable of simultaneously handling and synchronizing large-scale visual data (image tensors) and textual data (padded integer sequences).
- Advanced Framework Utilization: Utilizing the TensorFlow/Keras Functional API to architect complex, non-linear neural network topologies, in particular, learning how to route multiple independent inputs into a single merge layer.
- Computational Resource Management: Overcoming hardware and memory training constraints by applying Transfer Learning techniques. Freezing the ResNet50 base and pre-extracting the R^{2048} visual feature vectors meant the same convolutional computation did not have to be repeated during every LSTM training step, which saved a significant amount of processing time.

5.3 Future Work

While the current implementation works as a solid baseline, there are a few changes that could meaningfully improve the quality of the generated captions:

- Dataset Scaling: The model was trained on the Flickr8k dataset, which, while highly curated, is relatively small for deep learning standards. Future iterations should migrate the pipeline to larger benchmark datasets, such as MS COCO (Microsoft Common Objects in Context), which contains hundreds of thousands of images. This scaling would meaningfully improve the model's vocabulary diversity and its ability to generalize to new objects.
- Visual Attention Mechanisms: The current ResNet50 encoder compresses the entire image into a single static feature vector using global average pooling. Future work should implement an Attention Mechanism between the CNN and LSTM. Instead of relying on a static summary, attention would allow the decoder to dynamically compute spatial weights, "looking" at specific regions of the image (e.g., focusing on a person's face while generating the word "smiling") at each time step of the sentence generation, which should help reduce these small, fine-grained mistakes.

6. References

The following resources were used to develop the architecture and evaluate the image captioning system described in this report:

[1] Project Implementation. *image_captioning_CNN_LSTM.ipynb*. Primary source code, data preprocessing pipeline, feature extraction, model architecture, training, and evaluation used for the image captioning internship project.

[2] Hodosh, M., Young, P., & Hockenmaier, J. (2013). Framing Image Description as a Ranking Task: Data, Models and Evaluation Metrics. *Journal of Artificial Intelligence Research*, 47, 853–899.

[3] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778.

[4] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.

[5] Papineni, K., Roukos, S., Ward, T., & Zhu, W. J. (2002). BLEU: A Method for Automatic Evaluation of Machine Translation. *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, 311–318.

[6] *TensorFlow and Keras Documentation*. Retrieved from https://www.tensorflow.org/api_docs/python/tf/keras